

Time & Space Complexity Analysis

Divide & Conquer Solution to the Skyline Problem

① Identification of the Algorithm: The program solves the skyline Problem. The input is a list of rectangular buildings, each gives as a triple (L, H, R) where L is the left x -coordinate, H the height & R the right x -coordinate. The output is the silhouette formed when all buildings are viewed from a distance, expressed as a sequence of key points (x, h) , where h is the height of the outline immediately to the right of x .

The solution follows the classical divide and conquer paradigm, structurally identical to merge sort: the set of buildings is split into two halves, the skyline of each is computed recursively, and the two partial sky lines are combined by a linear-time merge that takes the pointwise maximum two height profiles.

[2] How the Problem is Divided into Subproblems:

The recursion is driven purely by the index range of the building array, in the same manner as merge sort. The call $go(l_0, h_i)$ computes the skyline of the buildings stored at indices $l_0 \dots h_i - 1$. Given a range of size $k = h_i - l_0 > 1$, the algorithm sets

$$mid = l_0 + \lfloor (h_i - l_0) / 2 \rfloor$$

and recurses on the two halves $go(l_0, mid)$ and $go(mid, h_i)$ of sizes $\lfloor k/2 \rfloor$ and $\lfloor k/2 \rfloor$.

The split is therefore balanced by construction and independent of the coordinate values.

[3] Base case of the Recursion:

Empty range ($n=0$): the skyline is the

empty list. This arises only when the algorithm is invoked on an input containing no valid buildings.

Single building ($n=1$): for the building (L, H, R)

the skyline is the two-point list $[(L, H), (R, 0)]$
- the height rises to H at L and falls back to ground level at R .

Both base cases are produced in $O(1)$.

Cost of the combine step:

The merge is a two-pointer sweep over the sorted x -coordinates of a and b . Each iteration of the while loop advances i , or j or both and performs a fixed number of comparisons plus one max and one add. The two clean-up loops drain whatever remains. Consequently the merge performs at most $|a| + |b|$ iterations of constant cost.

$$\text{Merge}(a, b) = O(|a| + |b|) = O(2 \cdot n/2 + 2 \cdot n/2) = O(n)$$

The divide step is also cheap: computing $\text{mid} = l_0 + (h_1 - l_0)/2$ is $O(1)$.

[4.4] The recurrence relation:

Splitting at the midpoint produces two subproblems of size $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$.

Combining the three costs above:

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n), T(1) = O(1).$$

which is conventionally written as:

$$T(n) = 2 \cdot T(n/2) + O(n)$$

This is exactly the merge sort recurrence.

[4.5] Solution by the Master Theorem:

Compare with the standard form $T(n) = a \cdot T(n/b) + f(n)$, where here $a=2$, $b=2$ and $f(n) =$

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n.$$

Since $f(n) = O(n) = O(n \log^a n)$, case 2 of the Master Theorem applies, giving

$$T(n) = O(n \log^a n \cdot \log n) = O(n \log n)$$

[4.6] Verification by the recursion-tree method:

At depth i there are 2^i subproblems, each of size $n/2^i$, and each costs $c \cdot n/2^i$ for the merge.

Level i	Subproblems	Size each	Work per level
0	1	n	$c \cdot n$
1	2	$n/2$	$c \cdot n$
2	4	$n/4$	$c \cdot n$
$\log_2 n$	n	1	$c \cdot n$

The recursion stops when the size reaches 1 after $\log_2 n + 1$ levels. Summing

$$T(n) = \sum_{i=0}^{\log_2 n} c \cdot n = c \cdot n \cdot (\log_2 n + 1) = O(n \log n)$$

Verification by substitution (induction)

$T(n) \leq c \cdot n \cdot \log_2 n + c \cdot n$ for a suitable constant $c > 0$.

Base: for $n=1$, $T(1) \leq c \cdot 1 \cdot 0 + c = c$, which holds by choosing c at least as large as the constant base-case cost.

Inductive step:

$$T(n) \leq 2[c \cdot (n/2) \cdot \log_2 (n/2) + c(n/2)] + c \cdot n$$

$$= c \cdot n \cdot (\log_2 n - 1) + c \cdot n + c \cdot n$$

$$= c \cdot n \cdot \log_2 n + c \cdot n$$

which re-establishes the hypothesis,
so, $T(n) = O(n \log n)$.

5. Best, Average and Worst Case

An interesting property of this implementation is that `add()` function discards a key point whenever it repeats the previous height.

The intermediate skylines can therefore be much smaller than the $2n$ upper bound of Lemma 1 and the running time genuinely depends on the geometry of the point input, not only on n .

Case	Input shape	Behaviour	Complexity
Worst	Buildings pairwise disjoint or in a staircase pattern, e.g. (1,1,2), (3,2,4), (5,3,6)	No key point is ever suppressed, so every level of the tree really does $O(n)$ work	$O(n \log n)$
Average	Randomly generated buildings	A constant fraction of the points survives each merge; the per-level work stays $O(n)$	$O(n \log n)$
Best	All n buildings identical, or one very tall buildings covering all the others	Every partial skyline collapses to $O(1)$ points, so only the bottom level costs $O(n)$ and the levels above form a geometric series	$O(n)$

In every case the number of recursive calls is exactly $2n-1$. So the $O(n)$ lower bound can never be beaten. Since the worst case is the figure normally quoted, the algorithm is stated as $O(n \log n)$.

Space Complexity

Because the split is always at the midpoint, the recursion tree is perfectly balanced and its height is $\lceil \log_2 n \rceil$. The call stack therefore holds $O(\log n)$ frames. no risk stack overflow even for very large n .

6.2 Auxiliary heap space:

The subtle point is that b is only created after the entire left recursion has returned and been destroyed. At any instant, the live vectors are those along a single root-to-leaf path.

$$\sum_{i=0}^{\log_2 n} O(n/2^i) = O(n \cdot (1 + 1/2 + 1/4 + \dots)) = O(2n) = O(n)$$

So the peak auxiliary space of the recursion is linear, not $O(n \log n)$ - the geometric series is the key. Adding the other data structures:

Storage	Purpose	Space
Worst + (input string)	Bullhole input buffered in memory	$O(s)$
α (parsed integers)	All tokens	$O(n)$
L, H, R	Valid buildings	$O(n)$
Recursion vectors	Live partial skylines on one path	$O(n)$
Call stack	$\lceil \log_2 n \rceil$ frames of $O(1)$ each	$O(\log n)$
Total	-	$O(s+n)$

Reading the input with `cin >> in` in a streaming fashion would remove the $O(s)$ buffer and reduce the total to $O(n)$; as written, the buffer is the price paid for the tolerant free-form parser.

Summary Results

Time: $O(s + n \log n)$ overall $O(n \log n)$

space: $O(s + n)$ auxiliary, with $O(\log n)$ recursion depth.

Component	Operation	Time	Space
Input buffering	string accumulation	$O(s)$	$O(s)$
Parsing	single character sweep	$O(s)$	$O(N)$
Filtering	One pass over triples	$O(N)$	$O(n)$
add()	Append with duplicate suppression	$O(1)$	$O(1)$
Divide	Midpoint index computation	$O(1)$	$O(1)$
Conquer	Two recursive calls on $n/2$	$2T(n/2)$	$O(\log n)$
Combine	Two-pointer max-merge	$O(n)$	$O(n)$
$\mathcal{O}()$ total	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	$O(n)$
output	print $k \leq 2n$ key points	$O(n)$	$O(1)$

Comparison with Alternative Approaches

Approach	Idea	Time	Space
Brute force (Coordinate sweep)	For every distinct x , scan all buildings to find the maximum height	$O(n^2)$	$O(n)$
Sweep line with a multiset/max heap	Sort $2n$ endpoints, Insert height on entry and erase or exit	$O(n \log n)$	$O(n)$
Divide and conquer (this program)	Recursively merge two half-skylines by pointwise maximum	$O(n \log n)$	$O(n)$

pointwise maximum two height profiles.